SOLID Principles Fix

Identified Violations

1. Single Responsibility Principle (SRP) Violations

File: `UserService.ts`

- Service handles too many responsibilities: CRUD, authentication, account management
- Methods like `verifyPassword`, `incrementLoginAttempts`, `approve`, `reject` belong to different concerns

2. Interface Segregation Principle (ISP) Violations

File: `UserService.ts`

- `UserServiceInterface` only defines basic CRUD methods
- `UserService` implements many additional methods not in the interface
- Clients depend on methods they don't use

3. Dependency Inversion Principle (DIP) Violations

Files: Multiple services

- Services directly depend on concrete `firestore` implementation
- Direct dependency on `bcrypt` library
- No abstraction layer for external dependencies

4. Open/Closed Principle (OCP) Violations

File: `UserService.ts`

- Adding new user types requires modifying existing code
- Factory method in `User.fromFirestore` needs modification for new types

Solution Architecture

New Service Structure

```
services/
├── interfaces/
│   ├── IUserReadService.ts
│   ├── IUserWriteService.ts
│   ├── IAuthenticationService.ts
│   ├── IAccountManagementService.ts
│   └── IPasswordService.ts
├── implementations/
│   ├── UserReadService.ts
```

```
│   ├── UserWriteService.ts
│   ├── AuthenticationService.ts
│   ├── AccountManagementService.ts
│   └── PasswordService.ts
└── UserService.ts (Facade - delegates to specialized services)
```

Dependency Injection Container

```
config/
└── container.ts (IoC container for dependency injection)
```

Implementation

Step 1: Create Service Interfaces (ISP)

Separate interfaces for different concerns:

```typescript
// IUserReadService.ts
export interface IUserReadService {
  getById(id: string): Promise<User | null>;
  getByEmail(email: string): Promise<User | null>;
  list(filter?: UserFilter): Promise<User[]>;
}

// IUserWriteService.ts
export interface IUserWriteService {
  create(email: string, userType: UserType, profile: UserProfile):
Promise<User>;
  update(id: string, updates: UpdateUserRequest): Promise<User>;
  delete(id: string): Promise<void>;
}

// IAuthenticationService.ts
export interface IAuthenticationService {
  verifyCredentials(email: string, password: string): Promise<User>;
  recordLogin(userId: string, ipAddress?: string): Promise<void>;
  recordFailedLogin(userId: string): Promise<void>;
}

// IAccountManagementService.ts
export interface IAccountManagementService {
  approve(userId: string, approvedBy: string): Promise<void>;
  reject(userId: string, reason: string, rejectedBy: string): Promise<void>;
  suspend(userId: string): Promise<void>;
  activate(userId: string): Promise<void>;
  deactivate(userId: string): Promise<void>;
}
```

```typescript
// IPasswordService.ts
export interface IPasswordService {
  hash(password: string): Promise<string>;
  verify(password: string, hash: string): Promise<boolean>;
  update(userId: string, newPassword: string): Promise<void>;
}
```

Step 2: Implement Specialized Services (SRP)

Each service has a single responsibility:

```typescript
// UserReadService.ts - Only handles reading users
export class UserReadService implements IUserReadService {
  constructor(
    private userRepository: IUserRepository,
    private cacheService?: ICacheService
  ) {}

  async getById(id: string): Promise<User | null> {
    // Check cache first
    if (this.cacheService) {
      const cached = await this.cacheService.get(`user:${id}`);
      if (cached) return cached;
    }

    // Fetch from repository
    const user = await this.userRepository.findById(id);

    // Cache result
    if (user && this.cacheService) {
      await this.cacheService.set(`user:${id}`, user, 300); // 5 minutes
    }

    return user;
  }

  async getByEmail(email: string): Promise<User | null> {
    return this.userRepository.findByEmail(email);
  }

  async list(filter?: UserFilter): Promise<User[]> {
    return this.userRepository.findAll(filter);
  }
}

// AuthenticationService.ts - Only handles authentication logic
export class AuthenticationService implements IAuthenticationService {
  constructor(
    private userReadService: IUserReadService,
    private passwordService: IPasswordService,
```

```
    private accountLockPolicy: IAccountLockPolicy
  ) {}

  async verifyCredentials(email: string, password: string): Promise<User> {
    const user = await this.userReadService.getByEmail(email);

    if (!user) {
      throw new AuthenticationError('Invalid credentials');
    }

    // Check if account is locked
    if (user.isAccountLocked()) {
      throw new AuthenticationError('Account is temporarily locked');
    }

    // Check if account can login
    if (!user.canLogin()) {
      throw new AuthenticationError('Account cannot login');
    }

    // Verify password
    const isValid = await this.passwordService.verify(password,
user.getPasswordHash());

    if (!isValid) {
      await this.recordFailedLogin(user.getId());
      throw new AuthenticationError('Invalid credentials');
    }

    return user;
  }

  async recordLogin(userId: string, ipAddress?: string): Promise<void> {
    const user = await this.userReadService.getById(userId);
    if (!user) throw new Error('User not found');

    user.recordLogin(ipAddress);
    await this.userRepository.update(user);
  }

  async recordFailedLogin(userId: string): Promise<void> {
    const user = await this.userReadService.getById(userId);
    if (!user) return;

    user.recordFailedLogin();

    // Apply lock policy
    if (this.accountLockPolicy.shouldLock(user)) {
      user.lockAccount(this.accountLockPolicy.getLockDuration());
    }

    await this.userRepository.update(user);
```

```
  }
}
```

Step 3: Create Repository Abstraction (DIP)

Abstract repository interface:

```
// IUserRepository.ts
export interface IUserRepository {
  findById(id: string): Promise<User | null>;
  findByEmail(email: string): Promise<User | null>;
  findByFirebaseUid(firebaseUid: string): Promise<User | null>;
  findAll(filter?: UserFilter): Promise<User[]>;
  create(user: User, passwordHash: string): Promise<User>;
  update(user: User): Promise<void>;
  delete(id: string): Promise<void>;
  exists(email: string): Promise<boolean>;
}

// FirestoreUserRepository.ts - Concrete implementation
export class FirestoreUserRepository implements IUserRepository {
  constructor(private firestore: FirebaseFirestore.Firestore) {}

  async findById(id: string): Promise<User | null> {
    const doc = await this.firestore.collection('users').doc(id).get();
    if (!doc.exists) return null;

    const data = doc.data();
    if (!data) return null;

    data.id = doc.id;
    return User.fromFirestore(data);
  }

  // ... other implementations
}
```

Step 4: Create Dependency Injection Container

```
// container.ts
import { IUserRepository } from './interfaces/IUserRepository';
import { IUserReadService } from './services/interfaces/IUserReadService';
import { IUserWriteService } from './services/interfaces/IUserWriteService';
import { IAuthenticationService } from
'./services/interfaces/IAuthenticationService';
import { IAccountManagementService } from
'./services/interfaces/IAccountManagementService';
import { IPasswordService } from './services/interfaces/IPasswordService';
```

```
import { FirestoreUserRepository } from
'./repositories/FirestoreUserRepository';
import { UserReadService } from './services/implementations/UserReadService';
import { UserWriteService } from './services/implementations/UserWriteService';
import { AuthenticationService } from
'./services/implementations/AuthenticationService';
import { AccountManagementService } from
'./services/implementations/AccountManagementService';
import { BcryptPasswordService } from
'./services/implementations/BcryptPasswordService';

export class Container {
  private static instance: Container;
  private services: Map<string, any> = new Map();

  private constructor() {
    this.registerServices();
  }

  static getInstance(): Container {
    if (!Container.instance) {
      Container.instance = new Container();
    }
    return Container.instance;
  }

  private registerServices(): void {
    // Register repositories
    this.services.set('IUserRepository', new
FirestoreUserRepository(firestore));

    // Register services with dependencies
    const userRepository = this.get<IUserRepository>('IUserRepository');
    const passwordService = new BcryptPasswordService();

    this.services.set('IPasswordService', passwordService);
    this.services.set('IUserReadService', new UserReadService(userRepository));
    this.services.set('IUserWriteService', new UserWriteService(userRepository,
passwordService));
    this.services.set('IAuthenticationService', new AuthenticationService(
      this.get('IUserReadService'),
      passwordService
    ));
    this.services.set('IAccountManagementService', new
AccountManagementService(
      this.get('IUserReadService'),
      userRepository
    ));
  }

  get<T>(serviceName: string): T {
    const service = this.services.get(serviceName);
```

```
    if (!service) {
      throw new Error(`Service ${serviceName} not registered`);
    }
    return service as T;
  }
}
```

Step 5: Refactor User Domain (OCP)

Make User class open for extension:

```
// User.ts - Abstract base class
export abstract class User extends Entity {
  // ... existing code

  // Factory method using registry pattern for OCP
  private static typeRegistry: Map<string, new (...args: any[]) => User> = new
Map();

  static registerType(role: string, constructor: new (...args: any[]) => User):
void {
    User.typeRegistry.set(role, constructor);
  }

  static fromFirestore(data: any): User {
    const Constructor = User.typeRegistry.get(data.role);
    if (!Constructor) {
      throw new Error(`Unknown user role: ${data.role}`);
    }

    // Use registered constructor
    const user = Constructor.createFromFirestore(data);
    return user;
  }

  static createFromFirestore(data: any): User {
    // Default implementation - to be overridden
    throw new Error('Must be implemented by subclass');
  }
}

// Register user types at startup
User.registerType(UserRole.CUSTOMER, CustomerUser);
User.registerType(UserRole.FARM_MANAGER, FarmStaffUser);
// ... register other types
```

Step 6: Update UserService as Facade

```
// UserService.ts - Facade pattern
export class UserService {
  constructor(
    private userReadService: IUserReadService,
    private userWriteService: IUserWriteService,
    private authService: IAuthenticationService,
    private accountService: IAccountManagementService
  ) {}

  async getUserById(id: string): Promise<User | null> {
    return this.userReadService.getById(id);
  }

  async createUser(email: string, userType: UserType, profile: UserProfile):
Promise<User> {
    return this.userWriteService.create(email, userType, profile);
  }

  async approveUser(userId: string, approvedBy: string): Promise<void> {
    return this.accountService.approve(userId, approvedBy);
  }

  // ... delegate other methods to specialized services
}
```

Step 7: Update Controllers to Use DI

```
// authControllerV2.ts - Updated to use dependency injection
import { Container } from '../config/container';
import { IUserReadService } from '../services/interfaces/IUserReadService';
import { IAuthenticationService } from
'../services/interfaces/IAuthenticationService';

const container = Container.getInstance();
const userReadService = container.get<IUserReadService>('IUserReadService');
const authService = container.get<IAuthenticationService>
('IAuthenticationService');

export const getMe = asyncHandler(async (req: Request, res: Response) => {
  const userId = (req as any).user?.id;

  if (!userId) {
    res.status(401).json({ success: false, message: 'Authentication required'
});
    return;
  }

  const user = await userReadService.getById(userId);
```

```
    if (!user) {
      res.status(404).json({ success: false, message: 'User not found' });
      return;
    }

    res.json({ success: true, data: { user } });
  });
```

Benefits of This Architecture

1. Single Responsibility Principle ☑

- Each service has one clear responsibility
- Easy to understand and maintain
- Changes are isolated to specific services

2. Open/Closed Principle ☑

- New user types can be added without modifying existing code
- Just register new type with `User.registerType()`
- Services are open for extension, closed for modification

3. Liskov Substitution Principle ☑

- All implementations can be substituted for their interfaces
- Mock implementations can be used for testing
- Services work with abstractions, not concretions

4. Interface Segregation Principle ☑

- Small, focused interfaces
- Clients only depend on methods they use
- No "fat" interfaces forcing unnecessary dependencies

5. Dependency Inversion Principle ☑

- High-level modules depend on abstractions
- Low-level modules implement abstractions
- Dependencies are injected, not instantiated
- Easy to swap implementations (e.g., switch from Firestore to PostgreSQL)

Migration Strategy

Phase 1: Create Interfaces

1. Create all service interfaces
2. Create repository interfaces
3. Keep existing code working

Phase 2: Implement Services

1. Implement specialized services
2. Create DI container
3. Update one controller at a time

Phase 3: Refactor Domain

1. Make User class extensible
2. Register user types
3. Update factory methods

Phase 4: Cleanup

1. Remove old monolithic UserService
2. Remove direct dependencies on firestore/bcrypt
3. Update tests to use mocks

Testing Benefits

```
// Easy to mock for testing
describe('AuthenticationService', () => {
  it('should verify credentials', async () => {
    const mockUserReadService = {
      getByEmail: jest.fn().mockResolvedValue(mockUser)
    };
    const mockPasswordService = {
      verify: jest.fn().mockResolvedValue(true)
    };

    const authService = new AuthenticationService(
      mockUserReadService,
      mockPasswordService
    );

    const result = await authService.verifyCredentials('test@example.com',
'password');
    expect(result).toBe(mockUser);
  });
});
```